Sir Syed CASE Institute of Technology

Lab #10

Artificial Intellegince

SUBMITTED BY:

UBAID AHMAD

ROLL NO:

2410-0011

SUBMITTED TO:

Ma’am Warda Asad

DATE:

03/05/2026

Lab TASKS:

Libraries

+ Code + Text

[1]

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score,
    confusion_matrix, ConfusionMatrixDisplay
)
```

Files

sample_data

Iris.csv

Load Dataset

[1]

```
df = pd.read_csv('Iris.csv')
print(df.head())
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

Check Missing Values

Check Missing Values

[3]

```
print(df.isnull().sum())
df.dropna(inplace=True)
```

```
Id               0
SepalLengthCm    0
SepalWidthCm     0
PetalLengthCm    0
PetalWidthCm     0
Species          0
dtype: int64
```

Prepare Features & Labels

[7]

```
encoder = LabelEncoder()
df['label'] = encoder.fit_transform(df['Species'])

X = df[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']].values
y = df['label'].values
class_names = encoder.classes_

print("Classes:", class_names)
```

```
Classes: ['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']
```

Train-Test Split

[8]

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=7)
print("Train size:", X_train.shape[0])
print("Test size :", X_test.shape[0])
```

```
Train size: 120
Test size : 30
```

Standardization

[9]

```
sc = StandardScaler()
X_train_sc = sc.fit_transform(X_train)
X_test_sc  = sc.transform(X_test)
```

KNN — Experimenting with K = 3, 5, 7

[10]

```
for k in [3, 5, 7]:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train_sc, y_train)
    preds = model.predict(X_test_sc)
    print(f"K={k}  |  Accuracy: {accuracy_score(y_test, preds)*100:.2f}%")
```

```
K=3  |  Accuracy: 86.67%
K=5  |  Accuracy: 86.67%
K=7  |  Accuracy: 93.33%
```

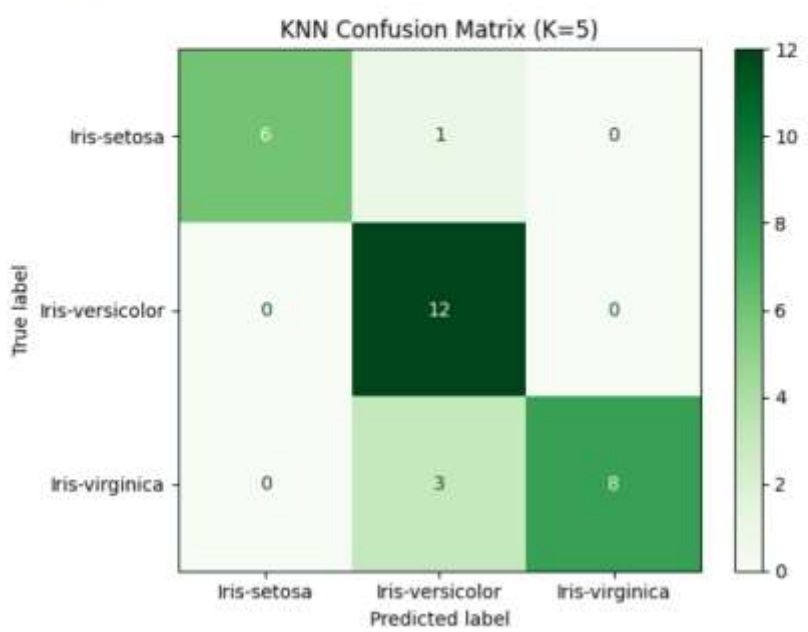
Final Model (K=5) — Full Evaluation

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_sc, y_train)
y_pred = knn.predict(X_test_sc)

print("Accuracy :", accuracy_score(y_test, y_pred)  * 100)
print("Precision:", precision_score(y_test, y_pred, average='macro') * 100)
print("Recall   :", recall_score(y_test, y_pred,    average='macro') * 100)
print("F1-Score :", f1_score(y_test, y_pred,        average='macro') * 100)
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Accuracy : 86.66666666666667
Precision: 91.66666666666666
Recall   : 86.14718614718613
F1-Score : 87.41083477925584

Confusion Matrix:
 [[ 6  1  0]
 [ 0 12  0]
 [ 0  3  8]]
```

Confusion Matrix — Plot

```
ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred,
    display_labels=class_names,
    cmap='Greens'
)
plt.title("KNN Confusion Matrix (K=5)")
plt.tight_layout()
plt.show()
```



END,,,